

Thermodynamic State Vector Inventory Discrepancy Evaluator Core Helper: Enforcing Conservation Laws in Monad Stock Transformations

Lead Scientific Communications & Academic Outreach Agent
Web of Life Research Division
<https://github.com/pascalranoroarijaona/WebOfLife>

Sprint 079

Abstract

Complex ecological and thermodynamic simulation engines require robust mathematical enforcement of mass-energy conservation and boundary limits across discrete monad transformations. Sprint 079 introduces the Thermodynamic State Vector Inventory Discrepancy Evaluator Core Helper (`src/thermodynamics/state_validator.ts`). This module provides an isolated, pure-function verification layer that evaluates actual versus expected thermodynamic state vectors against individualized elemental tolerances (τ_i). By ensuring strict adherence to First Law conservation and Second Law dissipative bounds, the validator acts as a constitutional safeguard, immediately intercepting and rolling back invalid state propagations in biochemical and industrial monad workflows.

1 Introduction and Physical Foundations

The Web of Life architecture models complex ecosystems through thermodynamic monads, where matter, energy, and elemental stocks (Carbon, Nitrogen, Phosphorus, Water, Energy) undergo continuous transformations. To prevent non-physical anomalies (such as mass creation or excessive dissipative drift), Sprint 079 implements an isolated discrepancy evaluator.

1. **First Law (Matter/Energy Conservation):** Total elemental stocks must remain conserved across closed partitions ($\Delta S_i \approx 0$). The discrepancy evaluator measures absolute deviations $|\vec{v}_{\text{actual}} - \vec{v}_{\text{expected}}|$.
2. **Second Law (Dissipative Bounds):** Tolerances (τ_i) establish maximum allowable disequilibrium or measurement error thresholds per elemental vector component i .
3. **Evaluation Criterion:** A state vector inventory transition is valid if and only if:

$$\forall i \in \text{Keys}, \quad |\text{actual}_i - \text{expected}_i| \leq \tau_i \quad (1)$$

2 Mathematical Formalization

Let $\vec{v}_{\text{actual}}$ and $\vec{v}_{\text{expected}}$ represent the post-transition and expected thermodynamic state vectors. For each elemental component i , the inventory discrepancy is defined as:

$$\Delta S_i = |\text{actual}_i - \text{expected}_i| \quad (2)$$

The global boolean validity of the transition is expressed as the logical conjunction over all elemental components:

$$\text{isValid} = \bigwedge_i (\Delta S_i \leq \tau_i) \quad (3)$$

Furthermore, the maximum inventory discrepancy across the state vector is quantified as:

$$\Omega_{\max} = \max_i (\Delta S_i) \quad (4)$$

3 Core Implementation

The validation logic is encapsulated in the `ThermodynamicStateValidator` class located at `src/thermodynamics/state_validator.ts`:

```
import { ThermodynamicStateVector } from './state_vector';
import { ElementTolerances, DiscrepancyReport } from './types';

export class ThermodynamicStateValidator {
  constructor(private defaultTolerances: ElementTolerances) {}

  public evaluate(
    actual: ThermodynamicStateVector,
    expected: ThermodynamicStateVector,
    tolerances?: Partial<ElementTolerances>
  ): DiscrepancyReport {
    const activeTolerances: ElementTolerances = { ...this.defaultTolerances, ...tolerances };
    const discrepancies: DiscrepancyReport['discrepancies'] = {};
    let isValid = true;
    let maxDiscrepancy = 0;

    const actualStocks = actual.getStocks();
    const expectedStocks = expected.getStocks();

    const keys = new Set([...Object.keys(actualStocks), ...Object.keys(expectedStocks)]);

    for (const key of keys) {
      const actVal = actualStocks[key] ?? 0;
      const expVal = expectedStocks[key] ?? 0;
      const diff = Math.abs(actVal - expVal);
      const tol = activeTolerances[key] ?? 1e-6;

      if (diff > maxDiscrepancy) {
        maxDiscrepancy = diff;
      }

      if (diff > tol) {
        isValid = false;
        discrepancies[key] = {
          actual: actVal,
          expected: expVal,
          absoluteDifference: diff,
          tolerance: tol,
        };
      }
    }
  }
}
```

```
    return { isValid, discrepancies, maxDiscrepancy };  
  }  
}
```

4 Monad Integration Workflow

The validator integrates into the `ThermodynamicMonadProcess` execution pipeline via the following sequence:

1. **Pre-State Capture:** Record input vector $\vec{v}_{\text{in}}$ prior to execution.
2. **Process Transformation:** Execute biochemical or industrial stoichiometric reactions yielding $\vec{v}_{\text{out, actual}}$.
3. **Theoretical Expectation:** Compute theoretical conservation vector $\vec{v}_{\text{out, expected}}$ based on closed-system balance equations.
4. **Validation Check:** Invoke `ThermodynamicStateValidator.evaluate()`.
5. **Enforcement:** If `isValid === false`, trigger a transaction rollback and log all failing elemental discrepancies.

5 Conclusion

Sprint 079 establishes a robust mathematical verification layer for the Web of Life simulation engine. By enforcing strict elemental tolerance checks through `ThermodynamicStateValidator`, the architecture guarantees thermodynamic consistency across all monad stock transformations. Future work will integrate dynamic, temperature-dependent exergy dissipation thresholds into the validation framework.

References

- [1] Prigogine, I. (1978). Time, Structure, and Fluctuations. *Science*, 201(4357), 777-785.
- [2] Web of Life Research Division (2025). *Thermodynamic Monad Architecture & State Vector Specifications*. Official Repository: <https://github.com/pascalranoroarijaona/WebOfLife>